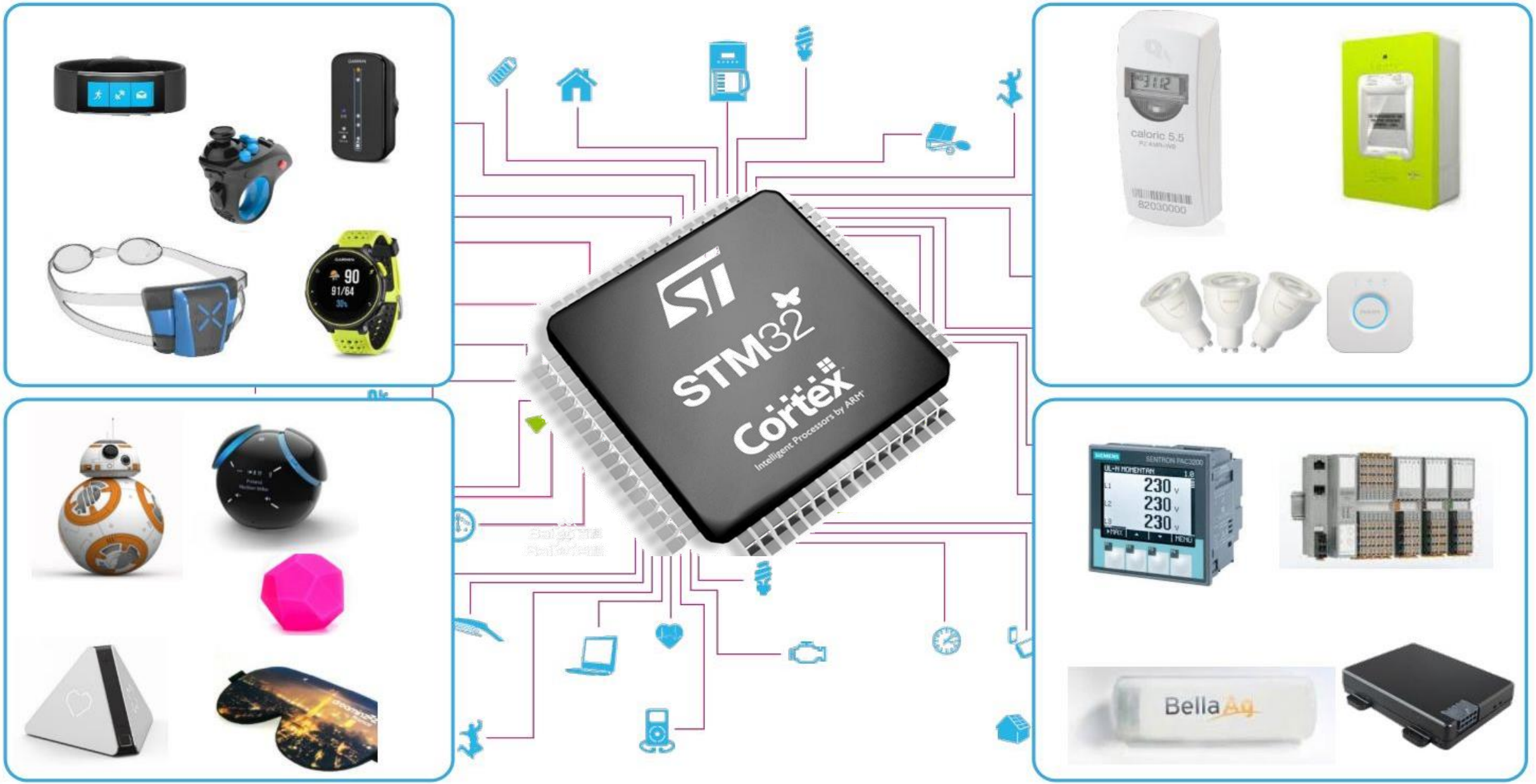
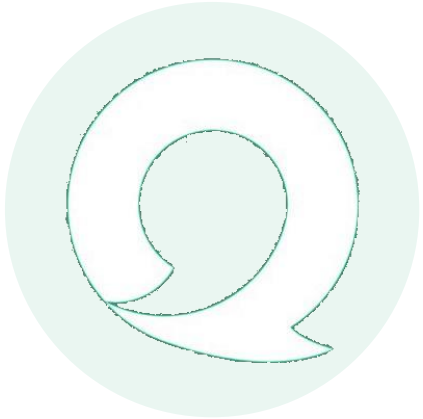


Instructor： WangXiaojing Email： wangxj@sustech.edu.cn



Lecture7

Timer-II

Topics

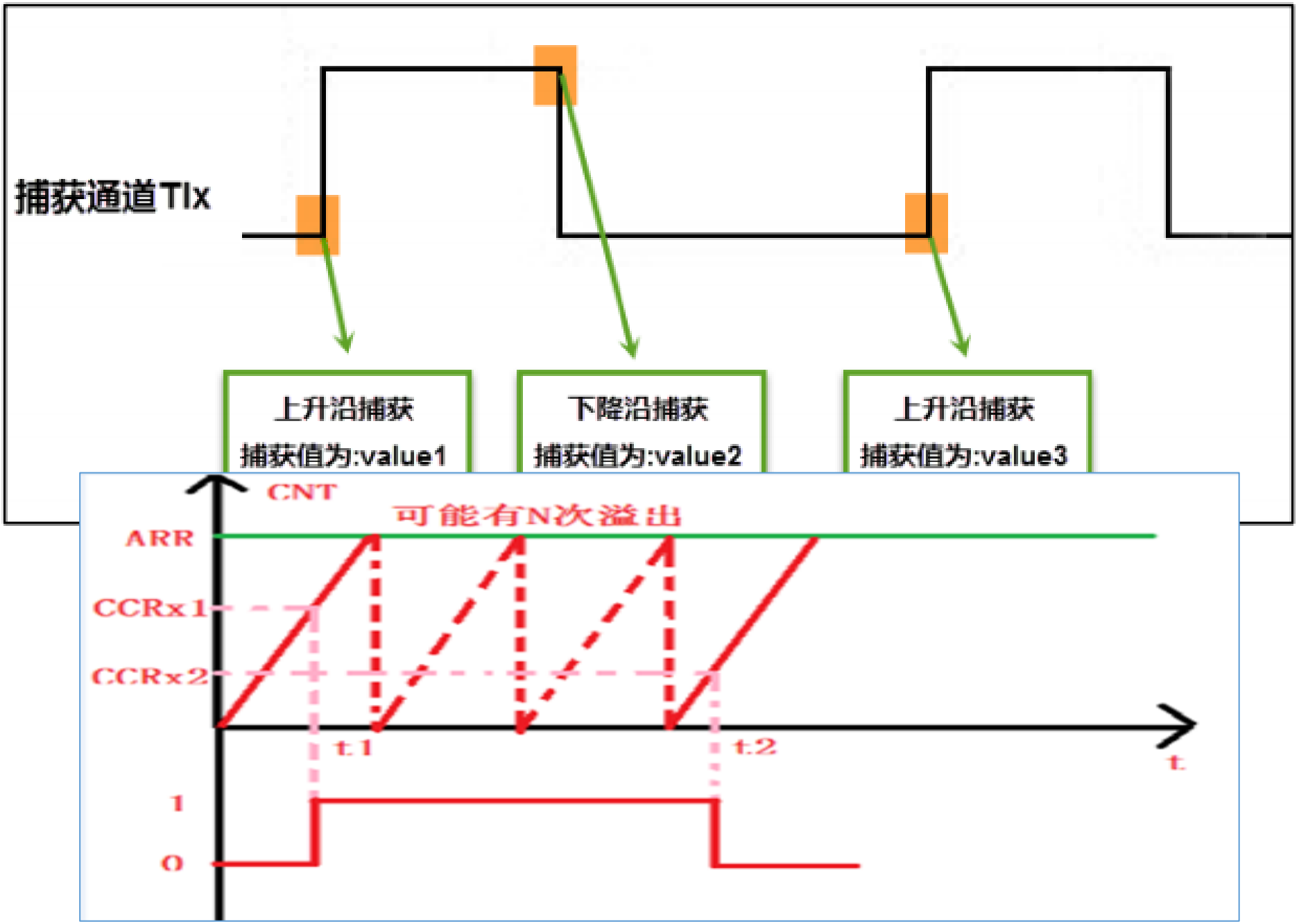
- General Timer--Input Capture
- General Timer--Output Compare(PWM Mode)

Part I: General Timer--Input Capture

General Description of Input Capture

As the name suggests, input capture is to capture the rising edge, falling edge or bilateral edge of the input signal. It is often used to measure the pulse width of the input signal, or the frequency and duty cycle of the input PWM signal.

The principle of input capture is simply that when the jump edge of the signal is captured, the value in the counter CNT is stored in the CCR register. The pulse width or frequency can be calculated by subtracting the values in the CCR register captured twice before and after. If the timer counter CNT overflows between two captures, that is, the captured pulse width time is too long, we need to do additional processing.


捕获通道TIx
上升沿捕获
捕获值为:value1
下降沿捕获
捕获值为:value2
上升沿捕获
捕获值为:value3
CNT
可能有N次溢出
ARR
CCRx1
CCRx2
t1
t2
t
1
0

Generally

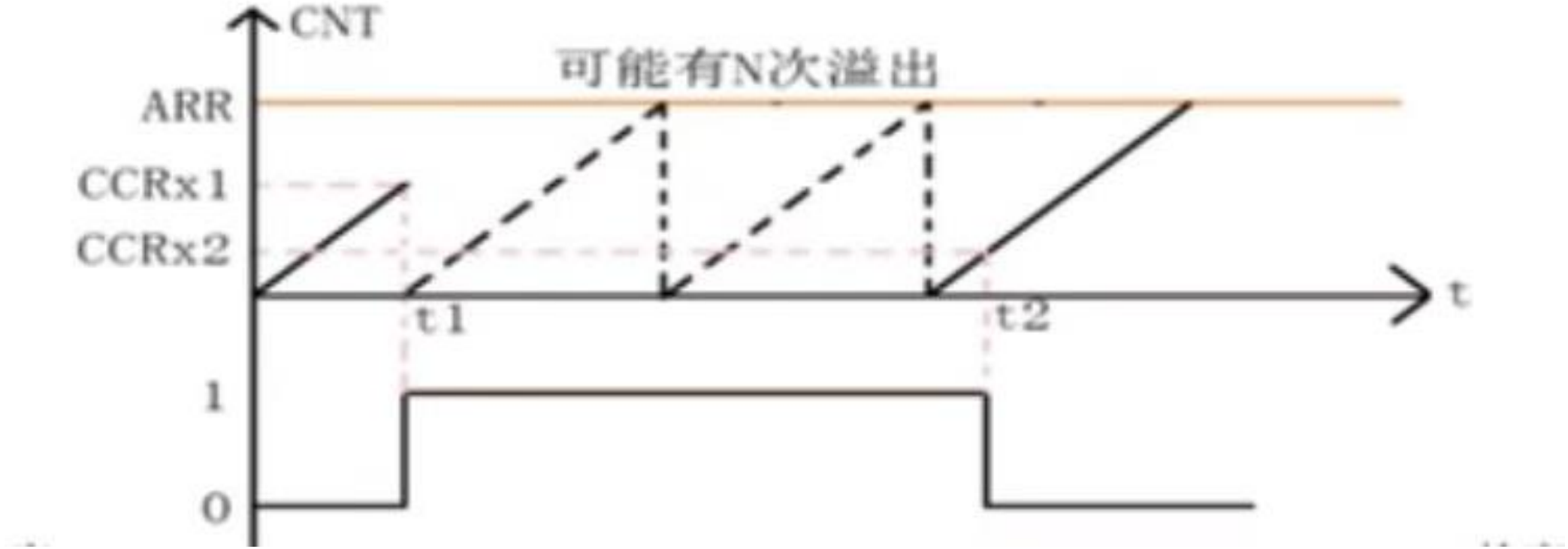
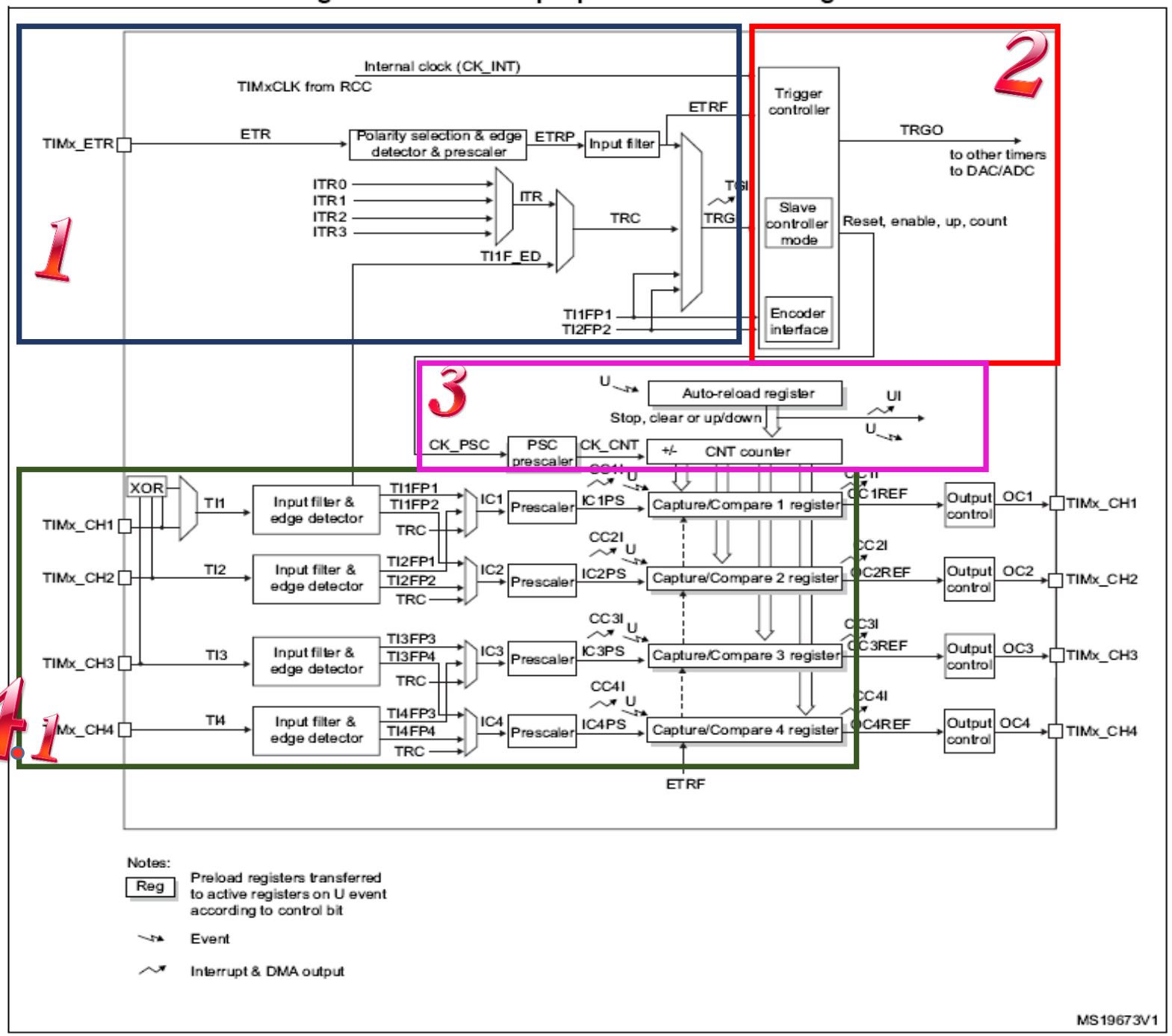


Figure 100. General-purpose timer block diagram



1

- 15.3.3 Clock selection
  - Internal clock source (CK_INT)
  - External clock source mode 1
  - External clock source mode 2

2

- 15.4.1 TIMx control register 1 (TIMx_CR1)
- 15.4.2 TIMx control register 2 (TIMx_CR2)
- 15.4.3 TIMx slave mode control register (TIMx_SMCR)

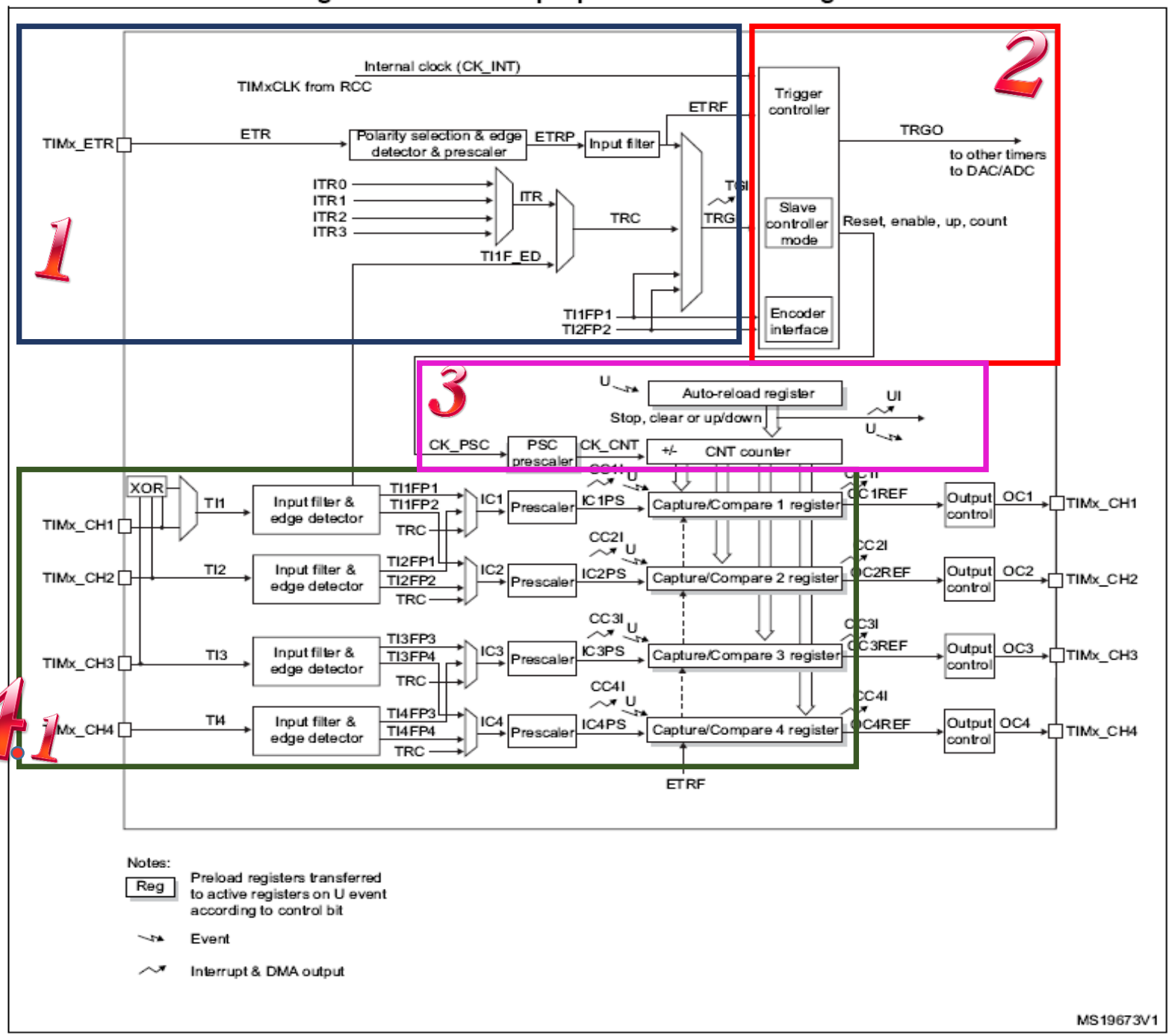
3

- 15.3.1 Time-base unit
- 15.3.2 Counter modes
  - Upcounting mode
  - Downcounting mode
  - Center-aligned mode (up/down counting)

4

- 15.3.4 Capture/compare channels
- 15.3.5 Input capture mode
- 15.3.6 PWM input mode
- 15.3.7 Forced output mode
- 15.3.8 Output compare mode
- 15.3.9 PWM mode
- 15.3.10 One-pulse mode
- 15.3.11 Clearing the OCxREF signal on an external event
- 15.3.12 Encoder interface mode
- 15.3.13 Timer input XOR function

Figure 100. General-purpose timer block diagram



- 15.4 TIMx registers
  - 15.4.1 TIMx control register 1 (TIMx_CR1)
  - 15.4.2 TIMx control register 2 (TIMx_CR2)
  - 15.4.3 TIMx slave mode control register (TIMx_SMCR)
  - 15.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)
  - 15.4.5 TIMx status register (TIMx_SR)
  - 15.4.6 TIMx event generation register (TIMx_EGR)
  - 15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)
  - 15.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)
  - 15.4.9 TIMx capture/compare enable register (TIMx_CCER)
  - 15.4.10 TIMx counter (TIMx_CNT)
  - 15.4.11 TIMx prescaler (TIMx_PSC)
  - 15.4.12 TIMx auto-reload register (TIMx_ARR)
  - 15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)
  - 15.4.14 TIMx capture/compare register 2 (TIMx_CCR2)
  - 15.4.15 TIMx capture/compare register 3 (TIMx_CCR3)
  - 15.4.16 TIMx capture/compare register 4 (TIMx_CCR4)
  - 15.4.17 TIMx DMA control register (TIMx_DCR)
  - 15.4.18 TIMx DMA address for full transfer (TIMx_DMAR)

15.3.5 Input capture mode

In Input capture mode, the Capture/Compare registers (TIMx_CCRx) are used to latch the value of the counter after a transition detected by the corresponding ICx signal. When a capture occurs, the corresponding CCXIF flag (TIMx_SR register) is set and an interrupt or a DMA request can be sent if they are enabled. If a capture occurs while the CCxIF flag was already high, then the over-capture flag CCxOF (TIMx_SR register) is set. CCxIF can be cleared by software by writing it to 0 or by reading the captured data stored in the TIMx_CCRx register. CCxOF is cleared when written to 0.

The following example shows how to capture the counter value in TIMx_CCR1 when TI1 input rises. To do this, use the following procedure:

- Select the active input: TIMx_CCR1 must be linked to the TI1 input, so write the CC1S bits to 01 in the TIMx_CCMR1 register. As soon as CC1S becomes different from 00, the channel is configured in input and the TIMx_CCR1 register becomes read-only.
- Program the needed input filter duration with respect to the signal connected to the timer (by programming the ICxF bits in the TIMx_CCMRx register if the input is one of the TIx inputs). Let's imagine that, when toggling, the input signal is not stable during at must five internal clock cycles. We must program a filter duration longer than these five clock cycles. We can validate a transition on TI1 when eight consecutive samples with the new level have been detected (sampled at f_{DTS} frequency). Then write IC1F bits to 0011 in the TIMx_CCMR1 register.
- Select the edge of the active transition on the TI1 channel by writing the CC1P bit to 0 in the TIMx_CCER register (rising edge in this case).
- Program the input prescaler. In our example, we wish the capture to be performed at each valid transition, so the prescaler is disabled (write IC1PS bits to 00 in the TIMx_CCMR1 register).
- Enable capture from the counter into the capture register by setting the CC1E bit in the TIMx_CCER register.
- If needed, enable the related interrupt request by setting the CC1IE bit in the TIMx_DIER register, and/or the DMA request by setting the CC1DE bit in the TIMx_DIER register.

When an input capture occurs:

- The TIMx_CCR1 register gets the value of the counter on the active transition.
- CC1IF flag is set (interrupt flag). CC1OF is also set if at least two consecutive captures occurred whereas the flag was not cleared.
- An interrupt is generated depending on the CC1IE bit.
- A DMA request is generated depending on the CC1DE bit.

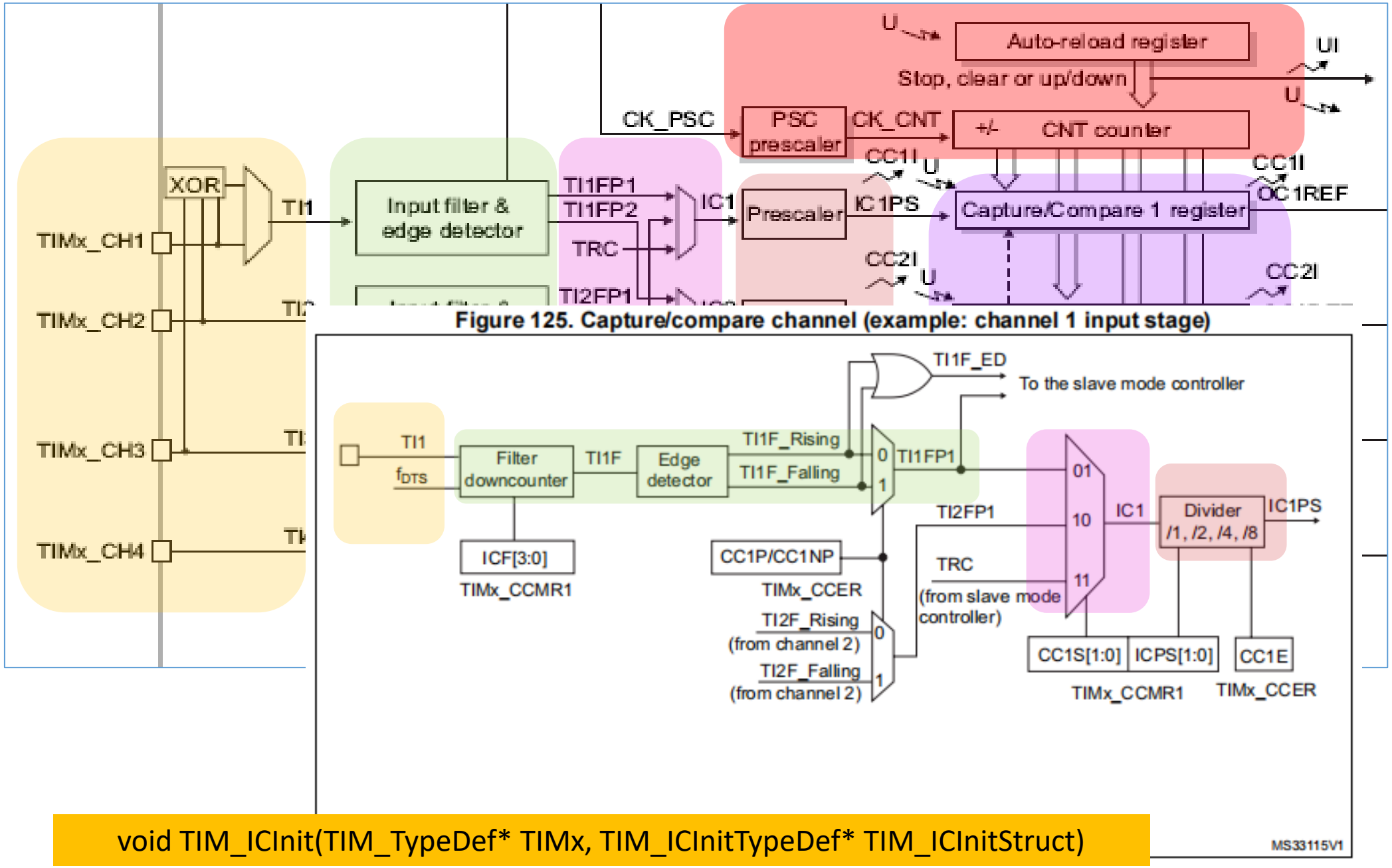


Figure 125. Capture/compare channel (example: channel 1 input stage)

void TIM_ICInit(TIM_TypeDef* TIMx, TIM_ICInitTypeDef* TIM_ICInitStruct)

void TIM_ICInit(TIM_TypeDef* TIMx, TIM_ICInitTypeDef* TIM_ICInitStruct)

```
115 typedef struct
116 {
117
118   uint16_t TIM_Channel;      /*!< Specifies the TIM channel.
119                                   This parameter can be a value of @ref TIM_Channel */
120
121   uint16_t TIM_ICPolarity;   /*!< Specifies the active edge of the input signal.
122                                   This parameter can be a value of @ref TIM_Input_Capture_Polarity */
123
124   uint16_t TIM_ICSelection;  /*!< Specifies the input.
125                                   This parameter can be a value of @ref TIM_Input_Capture_Selection */
126
127   uint16_t TIM_ICPrescaler;  /*!< Specifies the Input Capture Prescaler.
128                                   This parameter can be a value of @ref TIM_Input_Capture_Prescaler */
129
130   uint16_t TIM_ICFilter;     /*!< Specifies the input capture filter.
131                                   This parameter can be a number between 0x0 and 0xF */
132 } TIM_ICInitTypeDef;
```

General timer--Input Capture

1、**Enable timer clock**

RCC_APB1PeriphClockCmd();

2、**Timer Clock Configuration**

TIM_DeInit();
TIM_InternalClockConfig();

CK_INT，default setting，can be omitted

3、**Initialization the time base structure**

TIM_TimeBaseInit();

4、**Initialization the input capture structure**

TIM_ICInit();

5、**Timer interrupt configuration**

NVIC Configuration

TIM_ClearFlag(TIMx, **TIM_FLAG_Update|TIM_FLAG_CCx**);
void TIM_ITConfig(TIM5, **TIM_IT_Update|TIM_IT_CCx, ENABLE**);

6、**Enable timer**

TIM_Cmd();

7、**Write interrupt service function.**

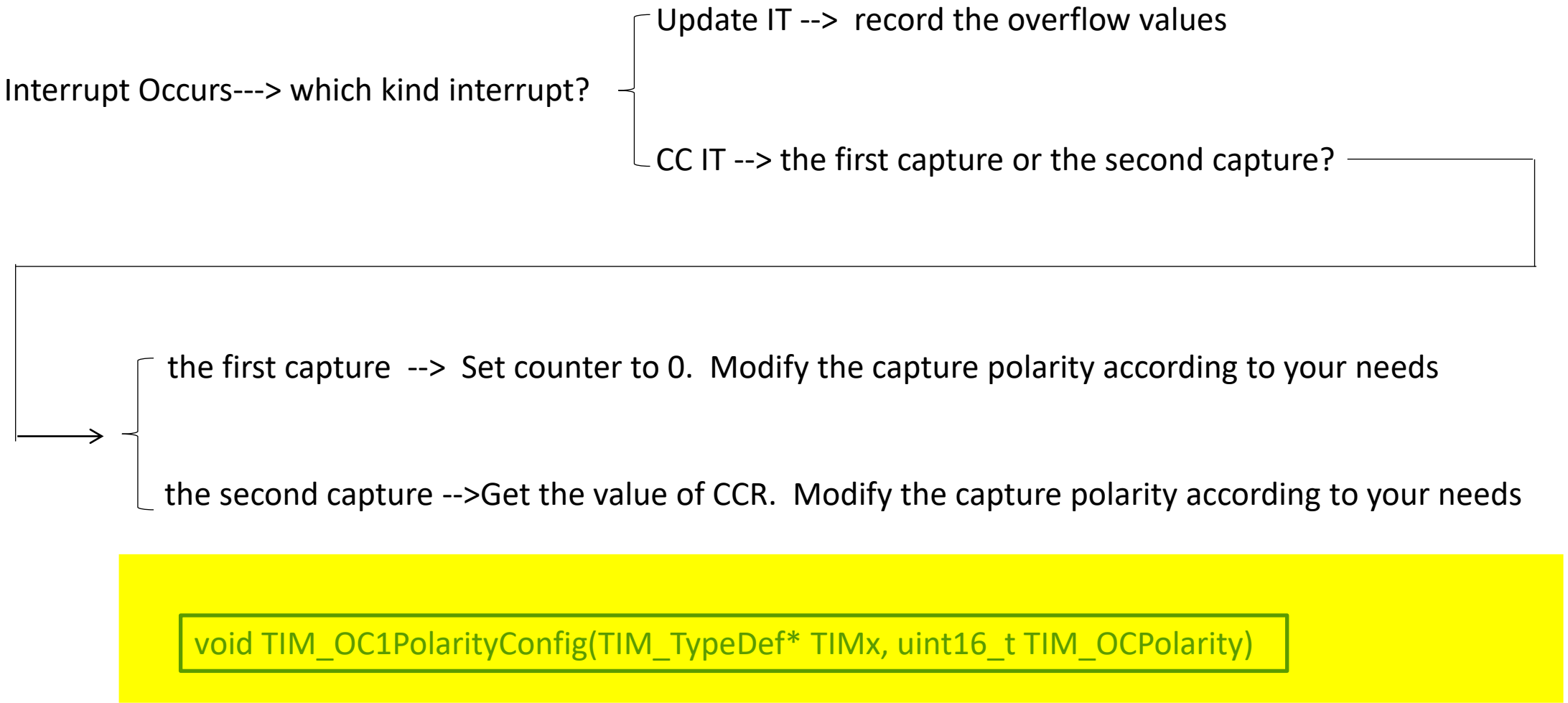
TIMx_IRQHandler();

0x4000 4000 - 0x4000 43FF	Reserved	APB1
0x4000 3C00 - 0x4000 3FFF	SPI3/I2S	
0x4000 3800 - 0x4000 3BFF	SPI2/I2S	
0x4000 3400 - 0x4000 37FF	Reserved	
0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)	
0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)	
0x4000 2800 - 0x4000 2BFF	RTC	
0x4000 2400 - 0x4000 27FF	Reserved	
0x4000 2000 - 0x4000 23FF	TIM14 timer	
0x4000 1C00 - 0x4000 1FFF	TIM13 timer	
0x4000 1800 - 0x4000 1BFF	TIM12 timer	
0x4000 1400 - 0x4000 17FF	TIM7 timer	
0x4000 1000 - 0x4000 13FF	TIM6 timer	
0x4000 0C00 - 0x4000 0FFF	TIM5 timer	
0x4000 0800 - 0x4000 0BFF	TIM4 timer	
0x4000 0400 - 0x4000 07FF	TIM3 timer	
0x4000 0000 - 0x4000 03FF	TIM2 timer	

TIM_GPIO_Config()
GPIO_Mode_IN_FLOATING

7、Write interrupt service function.

TIMx_IRQHandler();



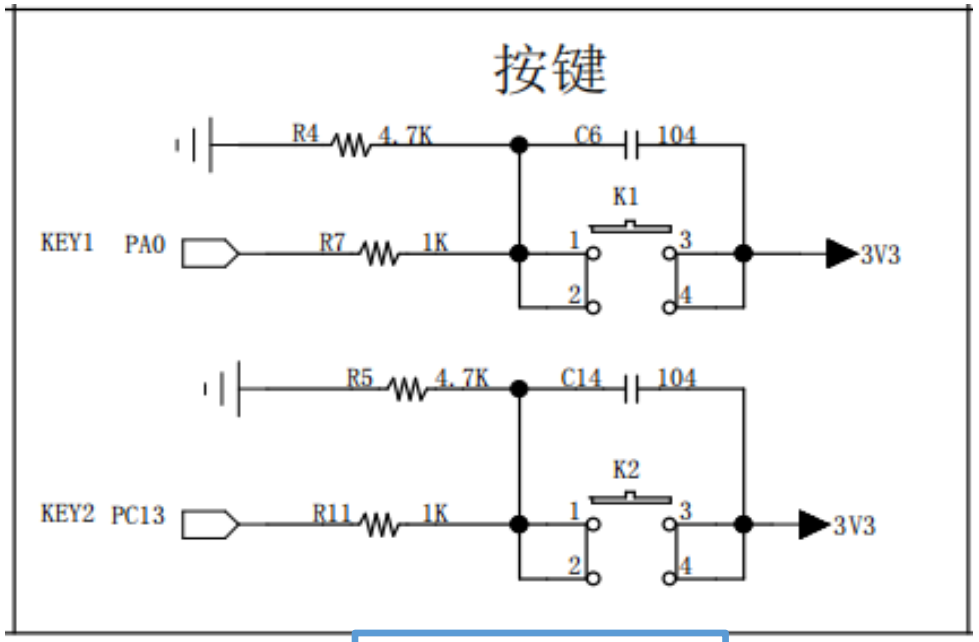
Homework1

- Write out the registers those are used(configurated or read) by the function

TIM_ICInit()

TIMx_IRQHandler()

and their meanings one by one ;



TIM5-CH1--> PA0

- Sort out the programme flow of **TIMx_IRQHandler()**.

- Modify the project "TIM-通用定时器-脉宽测量" to measure the frequency of the input signal（TIM4-CH1）。

Tips:

- ✓ modify the define xxxxx in GeneralTim.h
- ✓ delete one/two sentences in stm32f10x_it.c
- ✓ how to caculate the output value(periodic time/frequency)

More

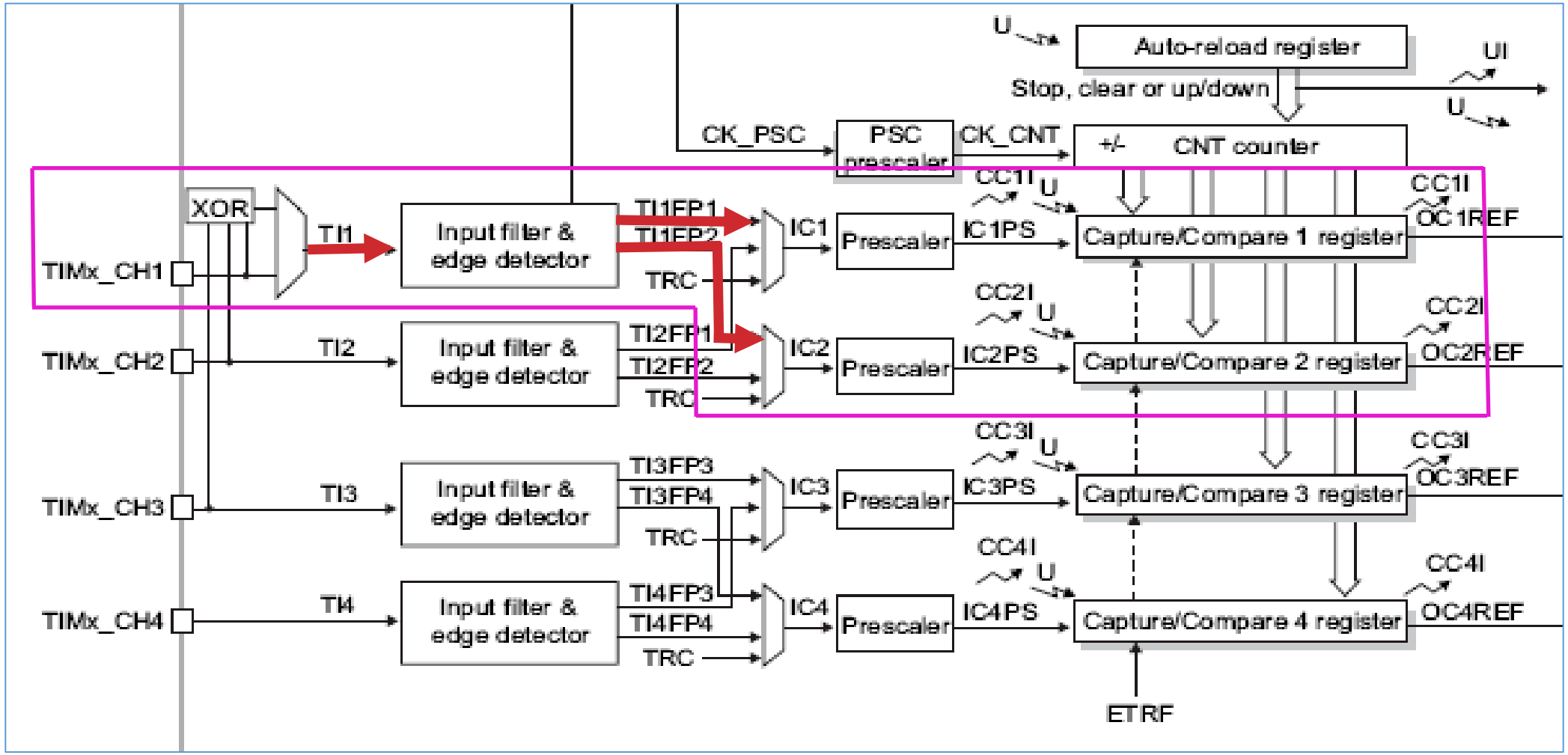
15.3.6 PWM input mode

This mode is a particular case of input capture mode. The procedure is the same except:

- Two ICx signals are mapped on the same TIx input.
- These 2 ICx signals are active on edges with opposite polarity.
- One of the two TIxFP signals is selected as trigger input and the slave mode controller is configured in reset mode.

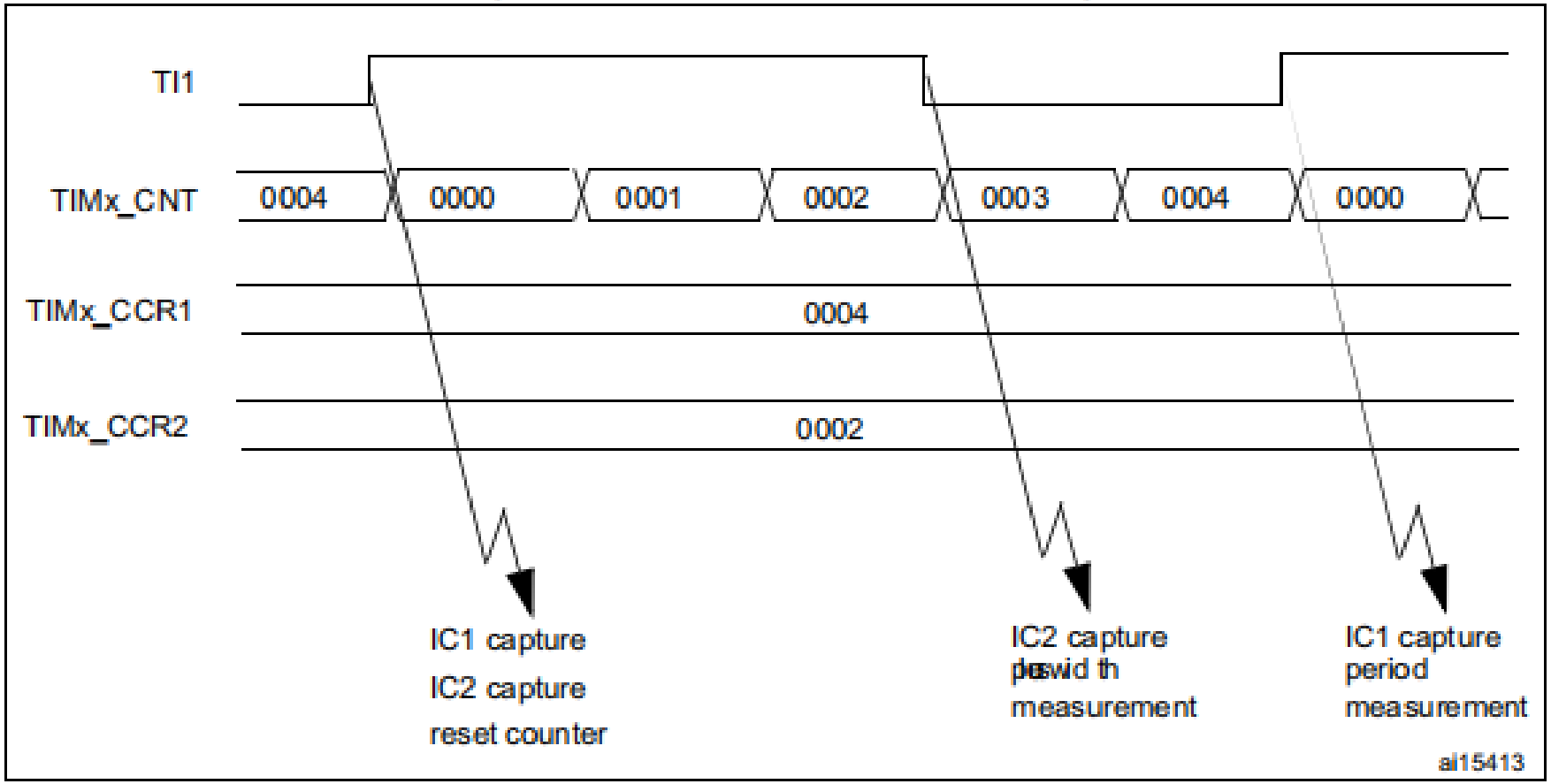
For example, the user can measure the period (in TIMx_CCR1 register) and the duty cycle (in TIMx_CCR2 register) of the PWM applied on TI1 using the following procedure (depending on CK_INT frequency and prescaler value):

- Select the active input for TIMx_CCR1: write the CC1S bits to 01 in the TIMx_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP1 (used both for capture in TIMx_CCR1 and counter clear): write the CC1P to '0' (active on rising edge).
- Select the active input for TIMx_CCR2: write the CC2S bits to 10 in the TIMx_CCMR1 register (TI1 selected).
- Select the active polarity for TI1FP2 (used for capture in TIMx_CCR2): write the CC2P bit to '1' (active on falling edge).
- Select the valid trigger input: write the TS bits to 101 in the TIMx_SMCR register (TI1FP1 selected).
- Configure the slave mode controller in reset mode: write the SMS bits to 100 in the TIMx_SMCR register.
- Enable the captures: write the CC1E and CC2E bits to '1 in the TIMx_CCER register.



The PWM input mode can be used only with the TIMx_CH1/TIMx_CH2 signals due to the fact that only TI1FP1 and TI2FP2 are connected to the slave mode controller.

Figure 128. PWM input mode timing



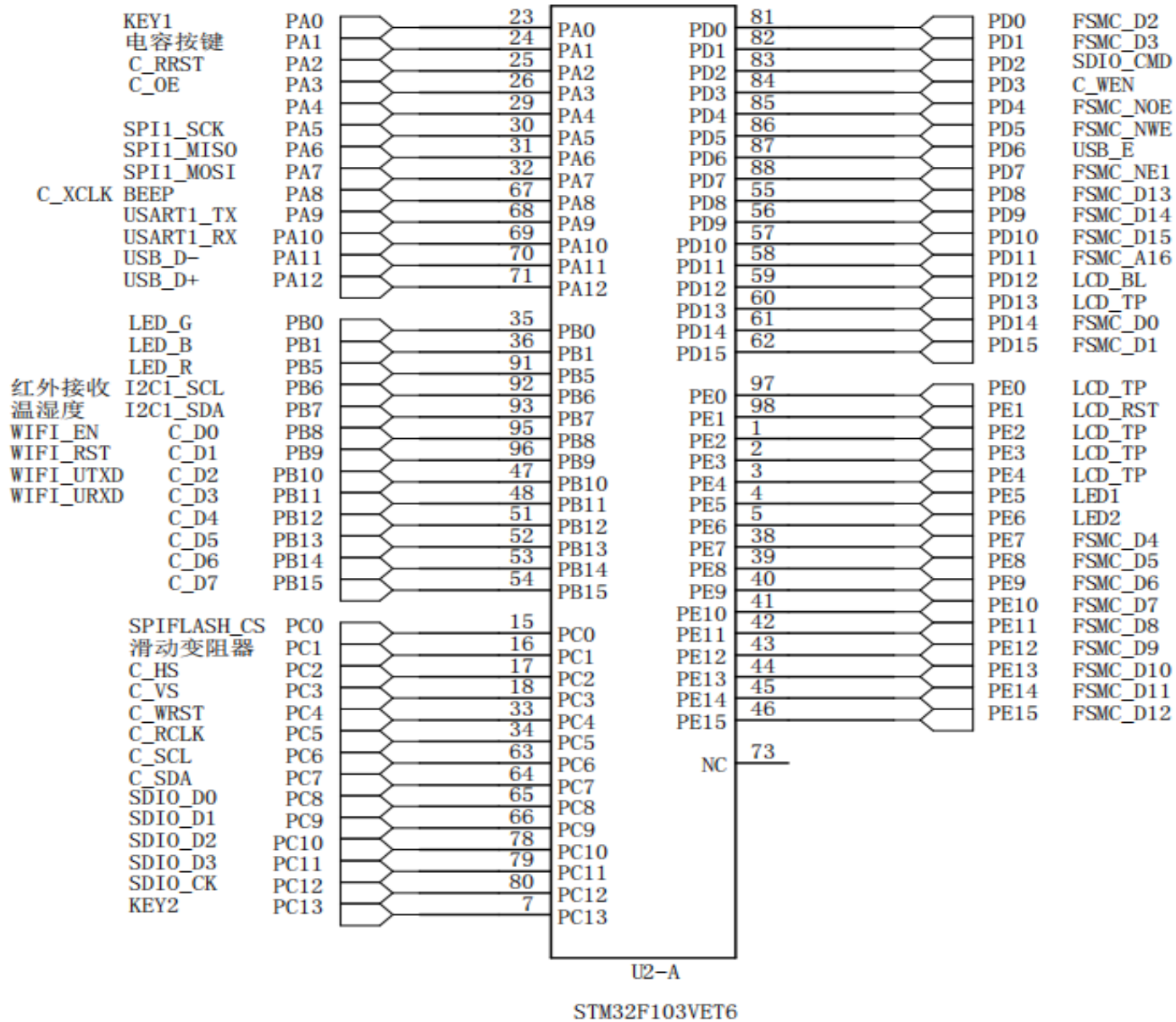
1. The PWM input mode can be used only with the TIMx_CH1/TIMx_CH2 signals due to the fact that only TI1FP1 and TI2FP2 are connected to the slave mode controller.

表 32-1 高级控制和通用定时器通道引脚分布

	高级定时器		通用定时器			
	TIM1	**TIM8**	**TIM2**	**TIM5**	**TIM3**	**TIM4**
CH1	PA8/PE9	PC6	PA0/PA15	PA0	PA6/PC6/PB4	PB6/PD12
CH1N	PB13/PA7/PE8	PA7				
CH2	PA9/PE11	PC7	PA1/PB3	PA1	PA7/PC7/PB5	PB7/PD13
CH2N	PB14/PB0/PE10	PB0				
CH3	PA10/PE13	PC8	PA2/PB10	PA2	PB0/PC8	PB8/PD14
CH3N	PB15/PB1/PE12	PB1				
CH4	PA11/PE14	PC9	PA3/PB11	PA3	PB1/PC9	PB9/PD15
ETR	PA12/PE7	PA0	PA0/PA15		PD2	PE0
BKIN	PB12/PA6/PE15	PA6				

MCU_GPIO_A

C_：表示Camera



Part II: General Timer--Output Compare (PWM Mode)

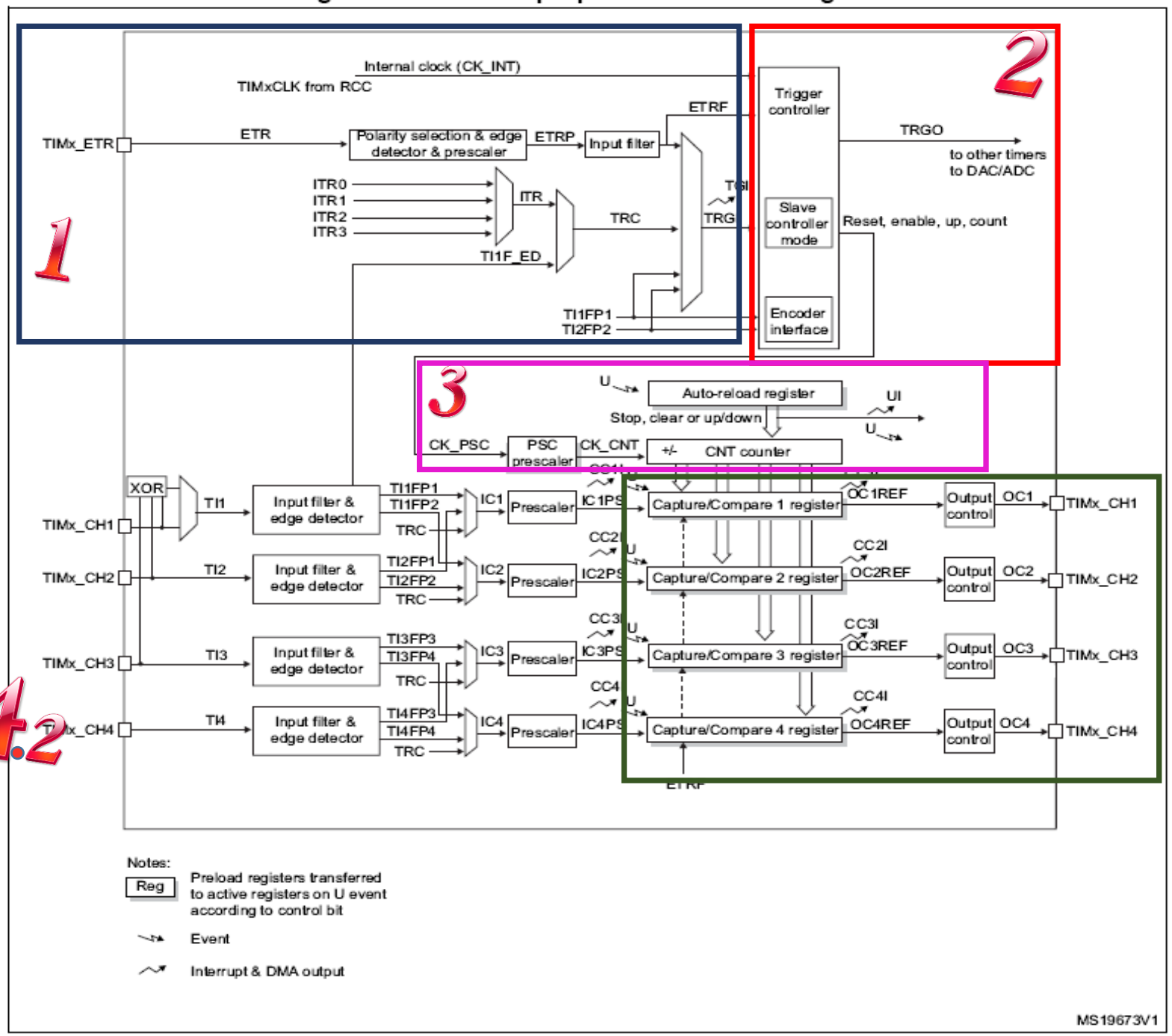
PWM

Pulse width modulation， is widely used in many fields ,eg. measurement, communication , power control , transformation.

Three typical applications of PWM：

- DC motor speed regulation：change the duty of PWM to change the effective voltage of DC motor, so as to adjust the speed of DC motor;
- Breathing LED ： change the duty of PWM to change the effective current flowing through the LED, so as to adjust the brightness of LED.
- Variable frequency and speed regulation：DC variable frequency and speed regulation, AC variable frequency and speed regulation--> Frequency Inverter-Based Air Conditioner

Figure 100. General-purpose timer block diagram



1

- 15.3.3 Clock selection
  - Internal clock source (CK_INT)
  - External clock source mode 1
  - External clock source mode 2

2

- 15.4.1 TIMx control register 1 (TIMx_CR1)
- 15.4.2 TIMx control register 2 (TIMx_CR2)
- 15.4.3 TIMx slave mode control register (TIMx_SMCR)

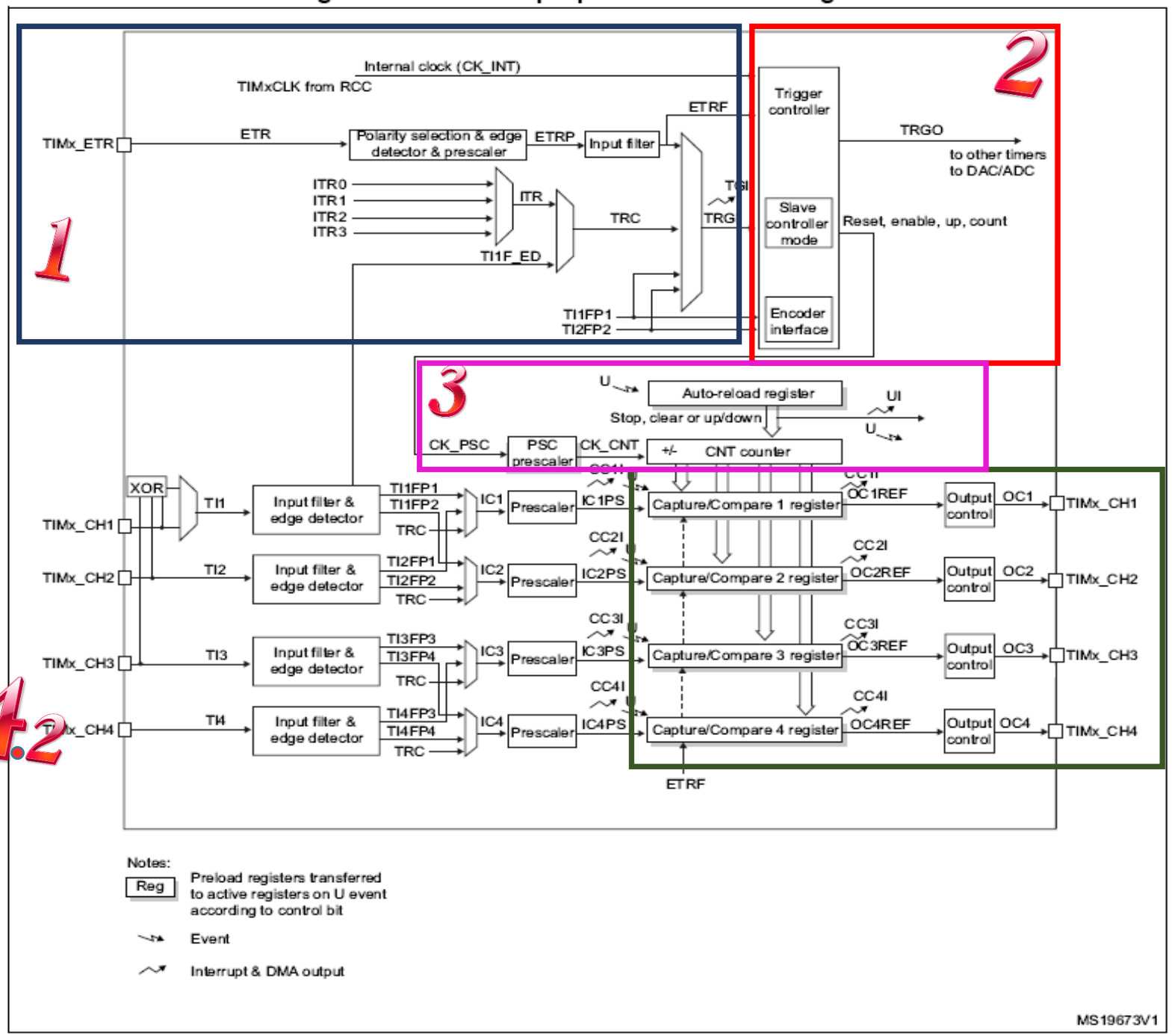
3

- 15.3.1 Time-base unit
- 15.3.2 Counter modes
  - Upcounting mode
  - Downcounting mode
  - Center-aligned mode (up/down counting)

4

- 15.3.4 Capture/compare channels
- 15.3.5 Input capture mode
- 15.3.6 PWM input mode
- 15.3.7 Forced output mode
- 15.3.8 Output compare mode
- 15.3.9 PWM mode
- 15.3.10 One-pulse mode
- 15.3.11 Clearing the OCxREF signal on an external event
- 15.3.12 Encoder interface mode
- 15.3.13 Timer input XOR function

Figure 100. General-purpose timer block diagram



- 15.4 TIMx registers
  - 15.4.1 TIMx control register 1 (TIMx_CR1)
  - 15.4.2 TIMx control register 2 (TIMx_CR2)
  - 15.4.3 TIMx slave mode control register (TIMx_SMCR)
  - 15.4.4 TIMx DMA/Interrupt enable register (TIMx_DIER)
  - 15.4.5 TIMx status register (TIMx_SR)
  - 15.4.6 TIMx event generation register (TIMx_EGR)
  - 15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)
  - 15.4.8 TIMx capture/compare mode register 2 (TIMx_CCMR2)
  - 15.4.9 TIMx capture/compare enable register (TIMx_CCER)
  - 15.4.10 TIMx counter (TIMx_CNT)
  - 15.4.11 TIMx prescaler (TIMx_PSC)
  - 15.4.12 TIMx auto-reload register (TIMx_ARR)
  - 15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)
  - 15.4.14 TIMx capture/compare register 2 (TIMx_CCR2)
  - 15.4.15 TIMx capture/compare register 3 (TIMx_CCR3)
  - 15.4.16 TIMx capture/compare register 4 (TIMx_CCR4)
  - 15.4.17 TIMx DMA control register (TIMx_DCR)
  - 15.4.18 TIMx DMA address for full transfer (TIMx_DMAR)

15.3.9 PWM mode

Pulse width modulation mode allows generating a signal with a frequency determined by the value of the TIMx_ARR register and a duty cycle determined by the value of the TIMx_CCRx register.

The PWM mode can be selected independently on each channel (one PWM per OCx output) by writing 110 (PWM mode 1) or ‘111 (PWM mode 2) in the OCxM bits in the TIMx_CCMRx register. The user must enable the corresponding preload register by setting the OCxPE bit in the TIMx_CCMRx register, and eventually the auto-reload preload register by setting the ARPE bit in the TIMx_CR1 register.

As the preload registers are transferred to the shadow registers only when an update event occurs, before starting the counter, the user has to initialize all the registers by setting the UG bit in the TIMx_EGR register.

OCx polarity is software programmable using the CCxP bit in the TIMx_CCER register. It can be programmed as active high or active low. OCx output is enabled by the CCxE bit in the TIMx_CCER register. Refer to the TIMx_CCERx register description for more details.

In PWM mode (1 or 2), TIMx_CNT and TIMx_CCRx are always compared to determine whether TIMx_CCRx≤ TIMx_CNT or TIMx_CNT≤ TIMx_CCRx (depending on the direction of the counter). However, to comply with the ETRF (OCREF can be cleared by an external event through the ETR signal until the next PWM period), the OCREF signal is asserted only:

- When the result of the comparison changes, or
- When the output compare mode (OCxM bits in TIMx_CCMRx register) switches from the “frozen” configuration (no comparison, OCxM=‘000) to one of the PWM modes (OCxM=‘110 or ‘111).

Figure 130. Edge-aligned PWM waveforms (ARR=8)

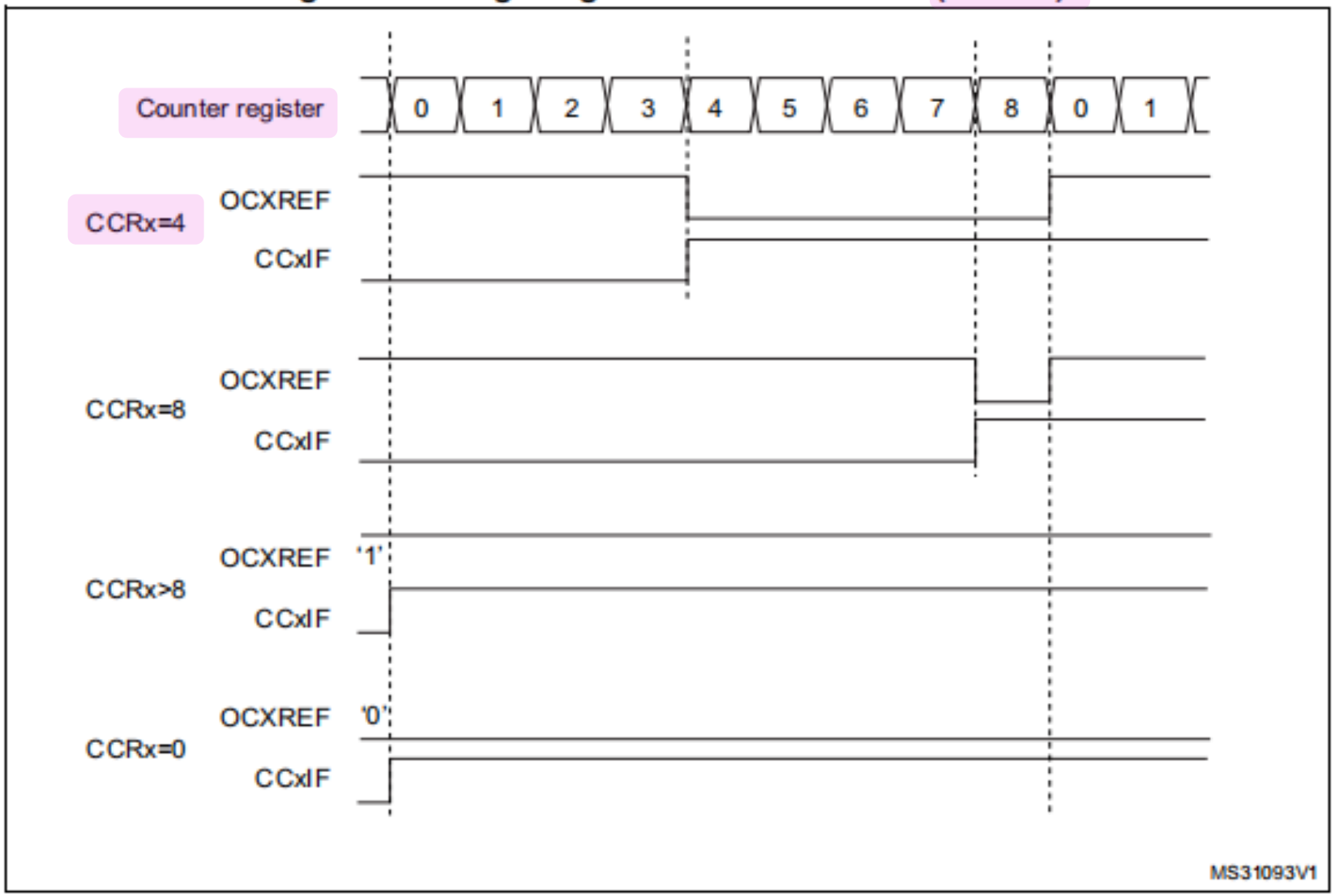
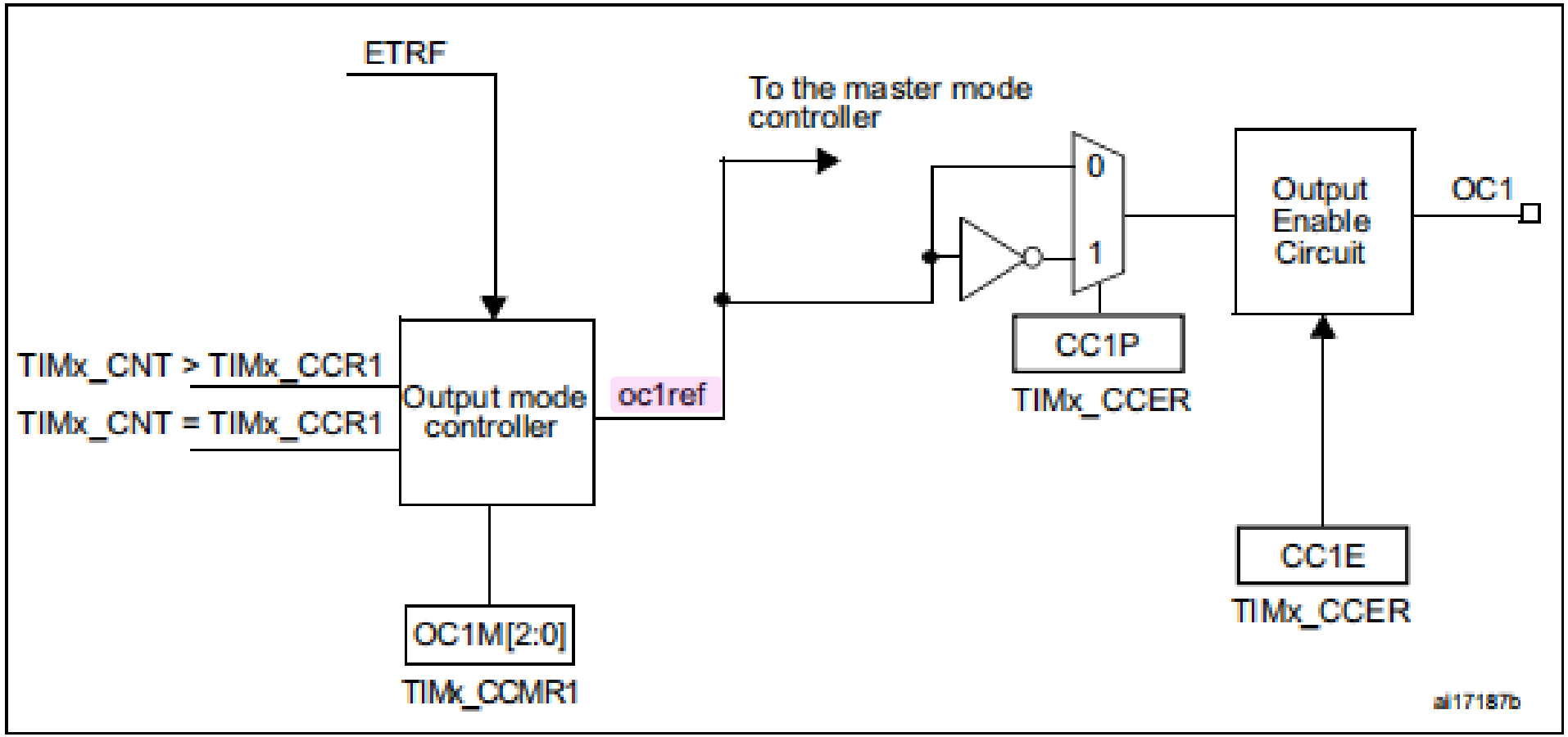


Figure 127. Output stage of capture/compare channel (channel 1)



15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)

Address offset: 0x18

Reset value: 0x0000

The channels can be used in input (capture mode) or in output (compare mode). The direction of a channel is defined by configuring the corresponding CCxS bits. All the other bits of this register have a different function in input and in output mode. For a given bit, OCxx describes its function when the channel is configured in output, ICxx describes its function when the channel is configured in input. Take care that the same bit can have a different meaning for the input stage and for the output stage.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 6:4 **OC1M**: Output compare 1 mode

These bits define the behavior of the output reference signal OC1REF from which OC1 and OC1N are derived. OC1REF is active high whereas OC1 and OC1N active level depends on CC1P and CC1NP bits.
000: Frozen - The comparison between the output compare register TIMx_CCR1 and the counter TIMx_CNT has no effect on the outputs.(this mode is used to generate a timing base).
001: Set channel 1 to active level on match. OC1REF signal is forced high when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1).
010: Set channel 1 to inactive level on match. OC1REF signal is forced low when the counter TIMx_CNT matches the capture/compare register 1 (TIMx_CCR1).
011: Toggle - OC1REF toggles when TIMx_CNT=TIMx_CCR1.
100: Force inactive level - OC1REF is forced low.
101: Force active level - OC1REF is forced high.
110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF='0) as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF=1).
111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive.

Note: In PWM mode 1 or 2, the OCREF level changes only when the result of the comparison changes or when the output compare mode switches from "frozen" mode to "PWM" mode.

110: PWM mode 1 - In upcounting, channel 1 is active as long as TIMx_CNT<TIMx_CCR1 else inactive. In downcounting, channel 1 is inactive (OC1REF= ‘0) as long as TIMx_CNT>TIMx_CCR1 else active (OC1REF=1).
111: PWM mode 2 - In upcounting, channel 1 is inactive as long as TIMx_CNT<TIMx_CCR1 else active. In downcounting, channel 1 is active as long as TIMx_CNT>TIMx_CCR1 else inactive.

表格 32-1 PWM1 与 PWM2 模式的区别

模式	计数器 CNT 计算方式	说明
PWM1	递增	CNT<CCR，通道 CH 为有效，否则为无效
	递减	CNT>CCR，通道 CH 为无效，否则为有效
PWM2	递增	CNT<CCR，通道 CH 为无效，否则为有效
	递减	CNT>CCR，通道 CH 为有效，否则为无效

15.4.9 TIMx capture/compare enable register (TIMx_CCER)

15		1	0
Res		CC1P	CC1E
		rw	rw

Bit 1 **CC1P**: Capture/Compare 1 output polarity

CC1 channel configured as output:

0: OC1 active high.

1: OC1 active low.

CC1 channel configured as input:

This bit selects whether IC1 or IC1 is used for trigger or capture operations.

0: non-inverted: capture is done on a rising edge of IC1. When used as external trigger, IC1 is non-inverted.

1: inverted: capture is done on a falling edge of IC1. When used as external trigger, IC1 is inverted.

Bit 0 **CC1E**: Capture/Compare 1 output enable

CC1 channel configured as output:

0: Off - OC1 is not active.

1: On - OC1 signal is output on the corresponding output pin.

CC1 channel configured as input:

This bit determines if a capture of the counter value can actually be done into the input capture/compare register 1 (TIMx_CCR1) or not.

0: Capture disabled.

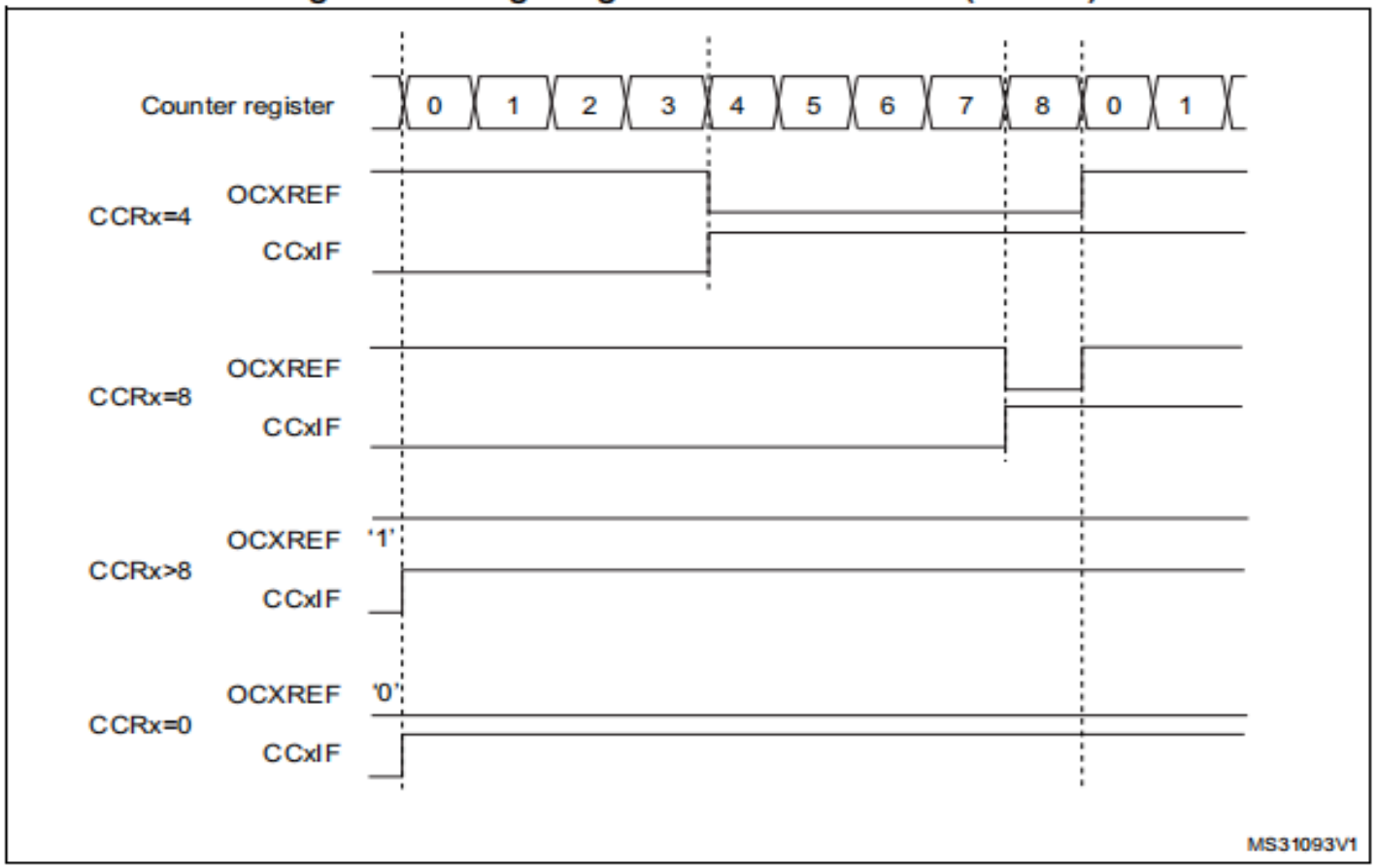
1: Capture enabled.

Table 87. Output control bit for standard OCx channels

CCxE bit	OCx output state
0	Output Disabled (OCx=0, OCx_EN=0)
1	OCx=OCxREF + Polarity, OCx_EN=1

In the following example, we consider PWM mode 1. The reference PWM signal OCxREF is high as long as TIMx_CNT <TIMx_CCRx else it becomes low. If the compare value in TIMx_CCRx is greater than the auto-reload value (in TIMx_ARR) then OCxREF is held at ‘1. If the compare value is 0 then OCxREF is held at ‘0. Figure 130 shows some edge-aligned PWM waveforms in an example where TIMx_ARR=8.

Figure 130. Edge-aligned PWM waveforms (ARR=8)



Frequency?

Duty?

void TIM_OCxInit(TIM_TypeDef* TIMx, TIM_OCInitTypeDef* TIM_OCInitStruct)

```
typedef struct
{
  uint16_t TIM_OCMode;        /*!< Specifies the TIM mode.
                                   This parameter can be a value of @ref TIM_Output_Compare_and_PWM_modes */

  uint16_t TIM_OutputState;   /*!< Specifies the TIM Output Compare state.
                                   This parameter can be a value of @ref TIM_Output_Compare_state */

  uint16_t TIM_OutputNState;  /*!< Specifies the TIM complementary Output Compare state.
                                   This parameter can be a value of @ref TIM_Output_Compare_N_state
                                   @note This parameter is valid only for TIM1 and TIM8. */

  uint16_t TIM_Pulse;         /*!< Specifies the pulse value to be loaded into the Capture Compare Register.
                                   This parameter can be a number between 0x0000 and 0xFFFF */

  uint16_t TIM_OCPolarity;    /*!< Specifies the output polarity.
                                   This parameter can be a value of @ref TIM_Output_Compare_Polarity */

  uint16_t TIM_OCNPolarity;   /*!< Specifies the complementary output polarity.
                                   This parameter can be a value of @ref TIM_Output_Compare_N_Polarity
                                   @note This parameter is valid only for TIM1 and TIM8. */

  uint16_t TIM_OCIdleState;   /*!< Specifies the TIM Output Compare pin state during Idle state.
                                   This parameter can be a value of @ref TIM_Output_Compare_Idle_State
                                   @note This parameter is valid only for TIM1 and TIM8. */

  uint16_t TIM_OCNIdleState;  /*!< Specifies the TIM Output Compare pin state during Idle state.
                                   This parameter can be a value of @ref TIM_Output_Compare_N_Idle_State
                                   @note This parameter is valid only for TIM1 and TIM8. */
} TIM_OCInitTypeDef;
```

15.4.7 TIMx capture/compare mode register 1 (TIMx_CCMR1)

Address offset: 0x18

Reset value: 0x0000

The channels can be used in input (capture mode) or in output (compare mode). The direction of a channel is defined by configuring the corresponding CCxS bits. All the other bits of this register have a different function in input and in output mode. For a given bit, OCxx describes its function when the channel is configured in output, ICxx describes its function when the channel is configured in input. Take care that the same bit can have a different meaning for the input stage and for the output stage.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15.4.9 TIMx capture/compare enable register (TIMx_CCER)

Address offset: 0x20

Reset value: 0x0000

15	14	13	12	11	10	9	
Reserved		CC4P	CC4E	Reserved		CC3P	C
		rw	rw			rw	

15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)

Address offset: 0x34

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro

General timer--Output Compare(PWM)

1、Enable timer clock

RCC_APB1PeriphClockCmd();

2、Timer Clock Configuration

TIM_DeInit();
TIM_InternalClockConfig();

CK_INT，default setting，can be omitted

3、Initialization the time base structure

TIM_TimeBaseInit();

4、Initialization the output compare structure

TIM_OCxInit();

5、Enable the Preload of TIMx-CCR& ARR

TIM_OC1PreloadConfig();
TIM_ARRPreloadConfig();

Optional

6、Enable timer

TIM_Cmd();

TIM_GPIO_Config()
GPIO_Mode_AF_PP

0x4000 4000 - 0x4000 43FF	Reserved	APB1
0x4000 3C00 - 0x4000 3FFF	SPI3/I2S	
0x4000 3800 - 0x4000 3BFF	SPI2/I2S	
0x4000 3400 - 0x4000 37FF	Reserved	
0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)	
0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)	
0x4000 2800 - 0x4000 2BFF	RTC	
0x4000 2400 - 0x4000 27FF	Reserved	
0x4000 2000 - 0x4000 23FF	TIM14 timer	
0x4000 1C00 - 0x4000 1FFF	TIM13 timer	
0x4000 1800 - 0x4000 1BFF	TIM12 timer	
0x4000 1400 - 0x4000 17FF	TIM7 timer	
0x4000 1000 - 0x4000 13FF	TIM6 timer	
0x4000 0C00 - 0x4000 0FFF	TIM5 timer	
0x4000 0800 - 0x4000 0BFF	TIM4 timer	
0x4000 0400 - 0x4000 07FF	TIM3 timer	
0x4000 0000 - 0x4000 03FF	TIM2 timer	

9.1.4 Alternate functions (AF)

It is necessary to program the Port Bit Configuration register before using a default alternate function.

- For alternate function inputs, the port must be configured in Input mode (floating, pull-up or pull-down) and the input pin must be driven externally.

Note: *It is also possible to emulate the AFI input pin by software by programming the GPIO controller. In this case, the port should be configured in Alternate Function Output mode. And obviously, the corresponding port should not be driven externally as it will be driven by the software using the GPIO controller.*

- For alternate function outputs, the port must be configured in Alternate Function Output mode (Push-Pull or Open-Drain).
- For bidirectional Alternate Functions, the port bit must be configured in Alternate Function Output mode (Push-Pull or Open-Drain). In this case the input driver is configured in input floating mode

If a port bit is configured as Alternate Function Output, this disconnects the output register and connects the pin to the output signal of an on-chip peripheral.

If software configures a GPIO pin as Alternate Function Output, but peripheral is not activated, its output is not specified.

9.1.11 GPIO configurations for device peripherals

Table 22 to *Table 33* give the GPIO configurations of the device peripherals.

Table 22. Advanced timers TIM1 and TIM8

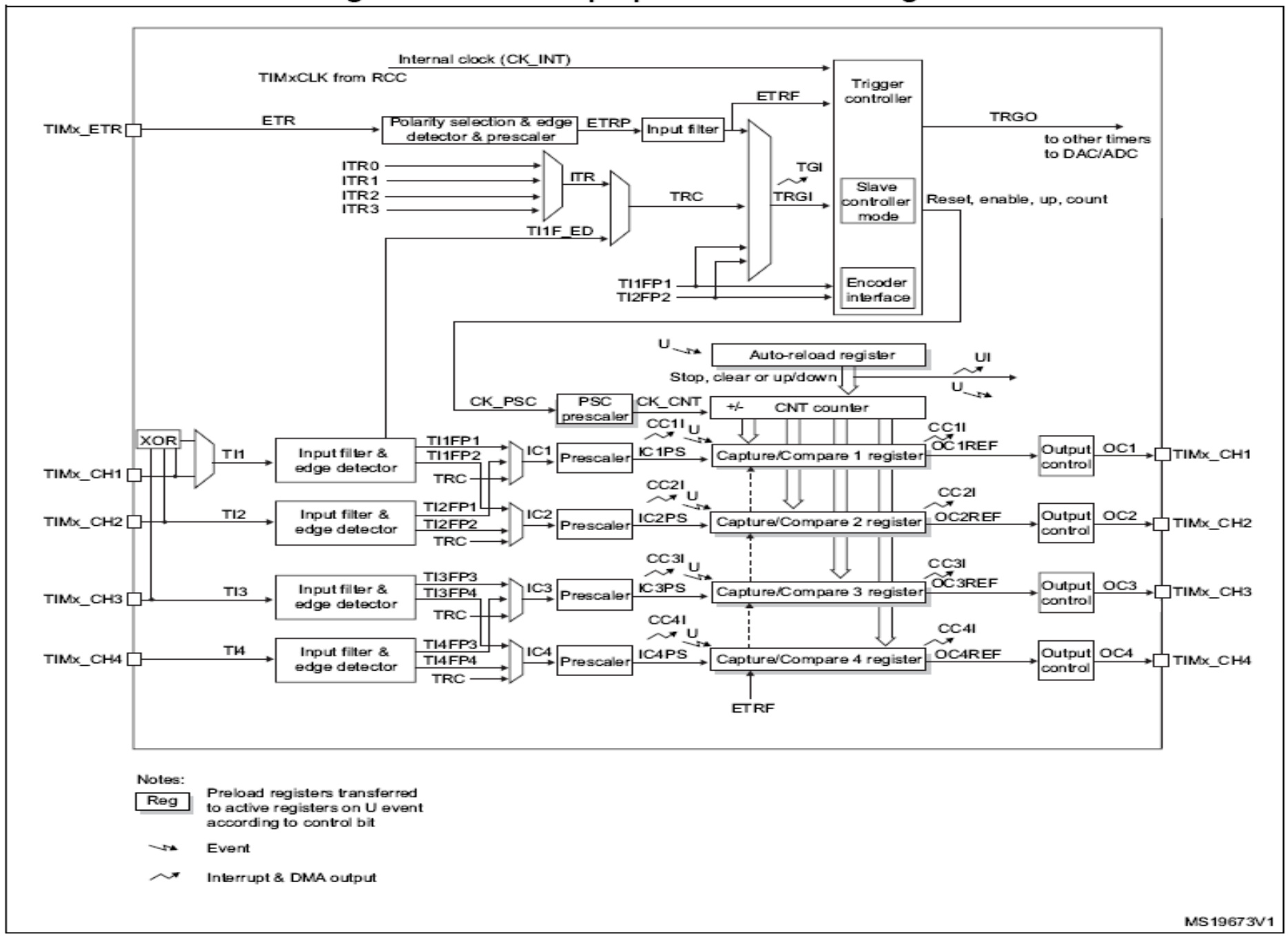
TIM1/8 pinout	Configuration	GPIO configuration
TIM1/8_CHx	Input capture channel x	Input floating
	Output compare channel x	Alternate function push-pull
TIM1/8_CHxN	Complementary output channel x	Alternate function push-pull
TIM1/8_BKIN	Break input	Input floating
TIM1/8_ETR	External trigger timer input	Input floating

Table 23. General-purpose timers TIM2/3/4/5

TIM2/3/4/5 pinout	Configuration	GPIO configuration
TIM2/3/4/5_CHx	Input capture channel x	Input floating
	Output compare channel x	Alternate function push-pull
TIM2/3/4/5_ETR	External trigger timer input	Input floating

```
//使能TIMx在CCR上的预装载寄
TIM_OC3PreloadConfig(TIM,TIM
//使能TIMx在ARR上的预装载寄
TIM_ARRPreloadConfig(TIM,EN
```

Figure 100. General-purpose timer block diagram



More

15.3.8 Output compare mode

This function is used to control an output waveform or indicating when a period of time has elapsed.

When a match is found between the capture/compare register and the counter, the output compare function:

- Assigns the corresponding output pin to a programmable value defined by the output compare mode (OCxM bits in the TIMx_CCMRx register) and the output polarity (CCxP bit in the TIMx_CCER register). The output pin can keep its level (OCXM=000), be set active (OCxM=001), be set inactive (OCxM=010) or can toggle (OCxM=011) on match.
- Sets a flag in the interrupt status register (CCxIF bit in the TIMx_SR register).
- Generates an interrupt if the corresponding interrupt mask is set (CCXIE bit in the TIMx_DIER register).
- Sends a DMA request if the corresponding enable bit is set (CCxDE bit in the TIMx_DIER register, CCDS bit in the TIMx_CR2 register for the DMA request selection).

The TIMx_CCRx registers can be programmed with or without preload registers using the OCxPE bit in the TIMx_CCMRx register.

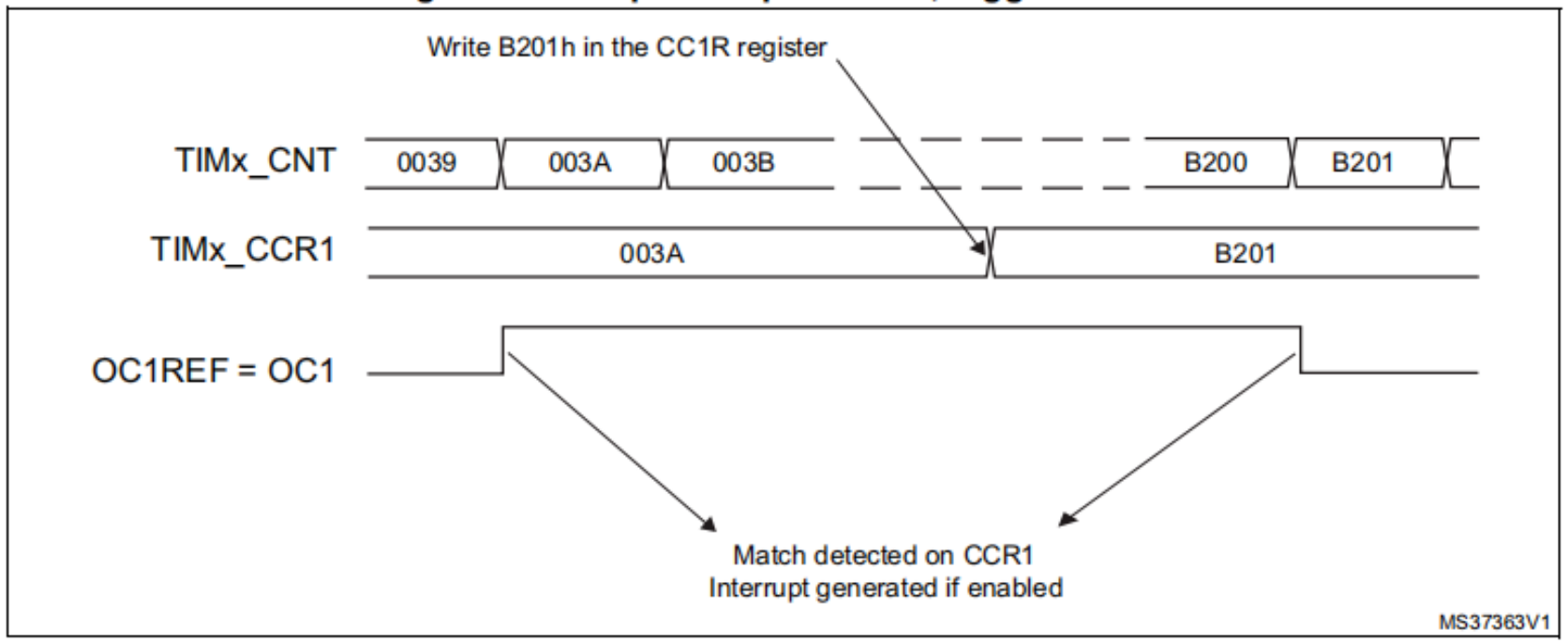
In output compare mode, the update event UEV has no effect on ocxref and OCx output. The timing resolution is one count of the counter. Output compare mode can also be used to output a single pulse (in One-pulse mode).

Procedure:

1. Select the counter clock (internal, external, prescaler).
2. Write the desired data in the TIMx_ARR and TIMx_CCRx registers.
3. Set the CCxIE and/or CCxDE bits if an interrupt and/or a DMA request is to be generated.
4. Select the output mode. For example, the user must write OCxM=011, OCxPE=0, CCxP=0 and CCxE=1 to toggle OCx output pin when CNT matches CCRx, CCRx preload is not used, OCx is enabled and active high.
5. Enable the counter by setting the CEN bit in the TIMx_CR1 register.

The TIMx_CCRx register can be updated at any time by software to control the output waveform, provided that the preload register is not enabled (OCxPE=0, else TIMx_CCRx shadow register is updated only at the next update event UEV). An example is given in *Figure 129*.

Figure 129. Output compare mode, toggle on OC1



Output Compare Mode VS PWM Mode

- Common: They can both be used to output PWM waveform, One TIM can output 4 PWM , and the duty cycle of each channel can be configured differrent.
- Difference:
  - —Output compare mode: can output four PWM with different frequencies and different duty cycles;
  - —PWM mode: can output four PWM with the same frequency and different duty cycle.

Homework2

- Write out the registers those are used(configurated or read) by the function **TIM_OCxInit() for General Tim** and their meanings one by one;
- Modify the frequency and duty of output PWM in the project"通用定时器-输出PWM" ----Easy

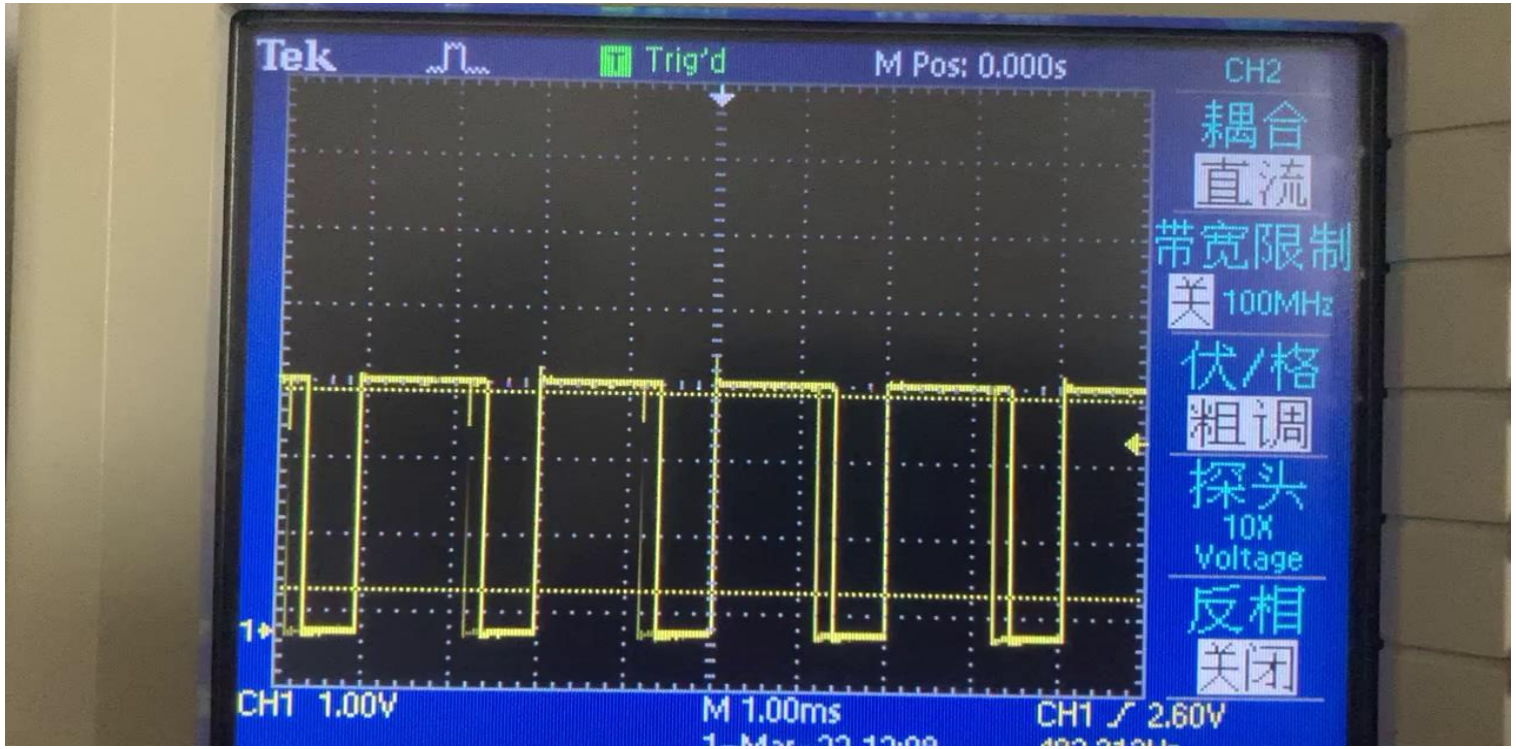
- Read the project"通用定时器-PWM呼吸灯-delay--示波器", Sort out the programme flow of this problem： How to constantly and regularly change the duty. You can observe the output wave by Oscilloscope(Which GPIO？)

```
void TIM_SetComparex(TIM_TypeDef* TIMx, uint16_t Compare1)
```

- Modify the project"通用定时器-PWM呼吸灯-delay--示波器", realize the breathing LED---Easy,change OCx-CH.

Optional

- Modify the project“通用定时器-PWM呼吸灯-delay--示波器”，use a TIM to realize timing instead of the dealy() function.


Tek
Trig'd
M Pos: 0.000s
CH2
耦合
直流
带宽限制
关
100MHz
伏/格
粗调
探头
10X
Voltage
反相
关闭
CH1 1.00V
M 1.00ms
CH1 ∕ 2.60V